

面向对象的设计

一、三个概念的关系

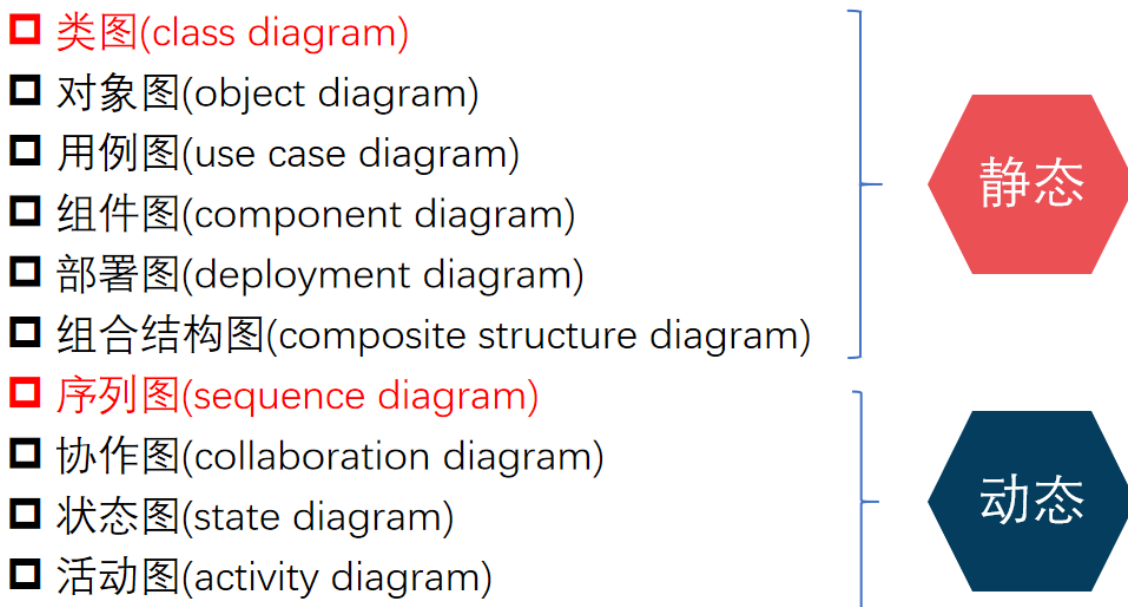
面向对象的四大基本特征是：抽象、封装、继承、多态。四大基本特征是我们在进行面向对象编程的准则，但是我们在进行面向对象编程之前还需要进行面向对象分析（Object-Oriented Analysis），通过分析业务，确定需求，找出对象，知道我们要干什么，然后就是进行面向对象的设计（Object-Oriented Design），知道我们如何干，最后才是面向对象的编程（Object-Oriented Program），具体的代码实现阶段。

二、为什么要面向对象设计（OOD）

不论是面向过程的思维方式，还是面向对象的思维方式，最终都是为了能更好的进行软件开发，而软件开发的唯一真理是“**软件一定会变化**”，而一个好的设计是应该能够轻松的应对变化，能被扩展，并且能够被复用。应该最好是**能以最小的代价满足变化，不用改变现有代码架构来满足扩展**，这样的设计才是一个优秀的设计，所以我们需要进行面向对象的设计。

三、UML语言

为了能够更好的进行面向对象的设计，所以需要引用专门的设计语言，也就是我们的UML语言，Unified Modeling Language，又称统一建模语言或标准建模语言，是始于1997年一个OMG(Object Management Group)标准，它是一个模型化和软件系统开发的图形化语言。UML为软件开发提供了一些标准的图例(10种)，统一开发思想，从而**促进团队协作**。这10中图例如下：



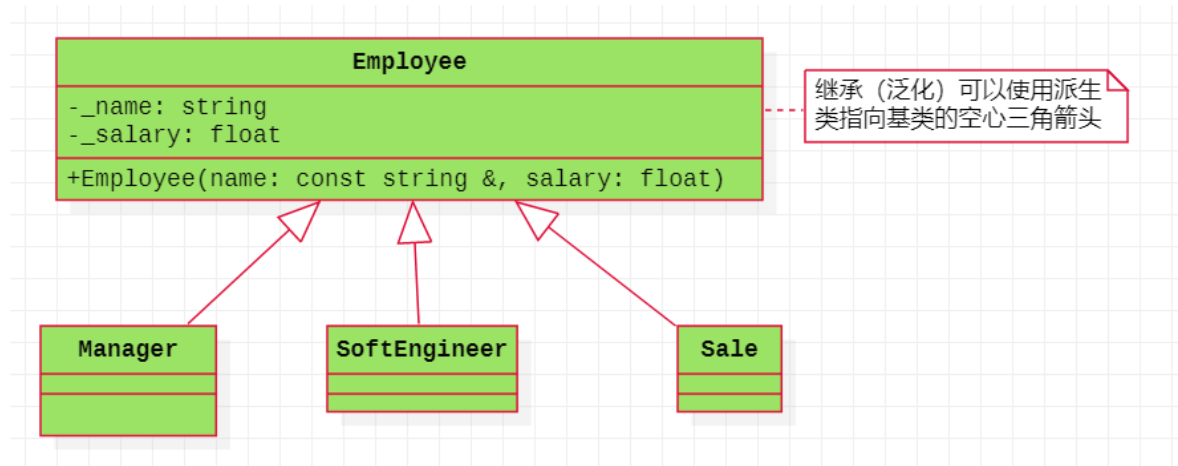
这里我们主要的来看红色的两种图例，第一种是类图，描述的是类与类之间的关系，第二种是一种动态图，可以看到一个动态的过程，后面对序列图我们在进一步了解，现在先来学习类图。

四、类与类之间的关系

我们知道UML中有10中图例，第一种就是类图，也就是描述类与类之间的关系，那类与类之间具备哪些关系呢？在此我们来看看他们之间的五种关系，继承（泛化）、关联、聚合、组合、依赖。

1、继承（泛化）

继承关系，或者泛化关系，描述的是两个类之间一种很强的关系，基类部分会成为派生类的一部分，在语义上是一个A is B的特点。例如经理是个员工、软件开发工程师是个员工、销售也是一个员工，所以可以设计下面的图例：



此例中，派生类经理、软件工程师、销售都是一个职员，体现出了is的关系。如果使用UML语言的话，使用**空心三角箭头从派生类指向基类**，这种关系其实是一种垂直的关系。

继承与泛化的区别：继承是先有基类，然后从基类产生新的类，也就是派生类；而泛化是先有派生类，然后多个派生类抽出基类，也就是先有派生类，然后才有基类。二者考虑的方向不一样，其实泛化更符合面向对象的思想。

2、关联

关联关系，是两个类之间最简单，最单纯的关系，表明两个类之间是有关系的，试想，如果两个类之间没有关系，那就不能满足面向对象的特点，对象与对象之间进行交互，使得彼此之间的状态发生变化的特点。而关联关系包括两种：双向的关联关系与单向的关联关系。但是不论哪种关联关系，两个类之间彼此并不负责对方的生命周期。注意在语义上是A has B的关系，一种固定的关系。在代码上的表现形式是：使用引用或者指针。两种关联关系的例子：

双向的关联关系（使用直线），客户与订单之间的关系。

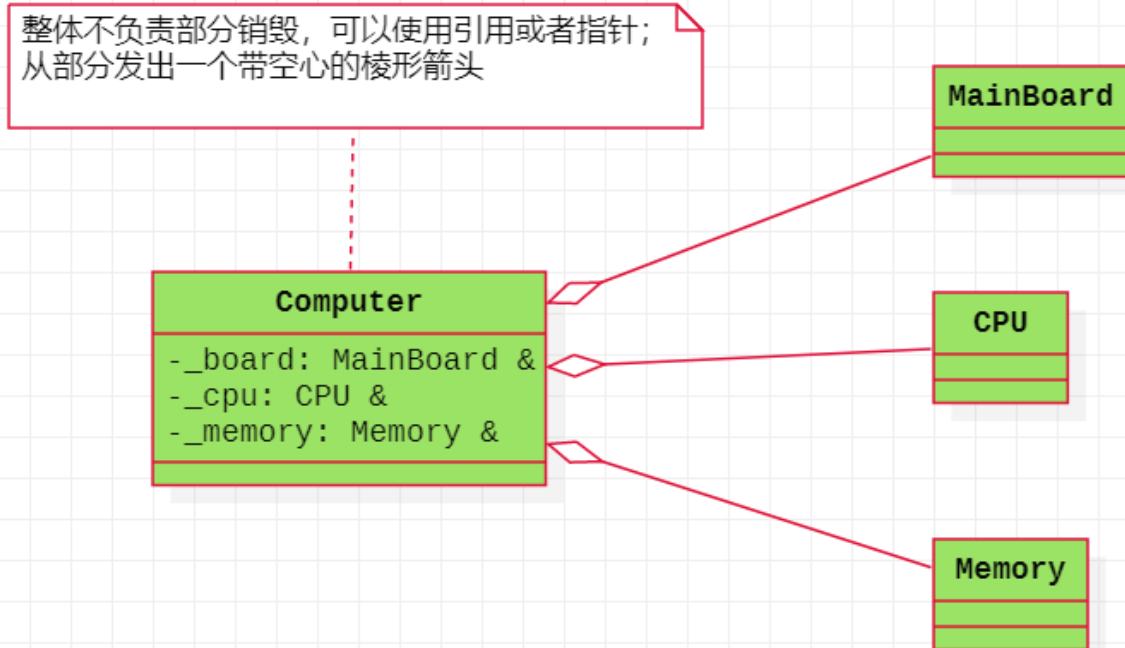


单向的关联关系，条件变量与互斥锁（Condition知道MutexLock的存在，但是MutexLock不知道Condition的存在，使用实线箭头从Condition指向MutexLock)



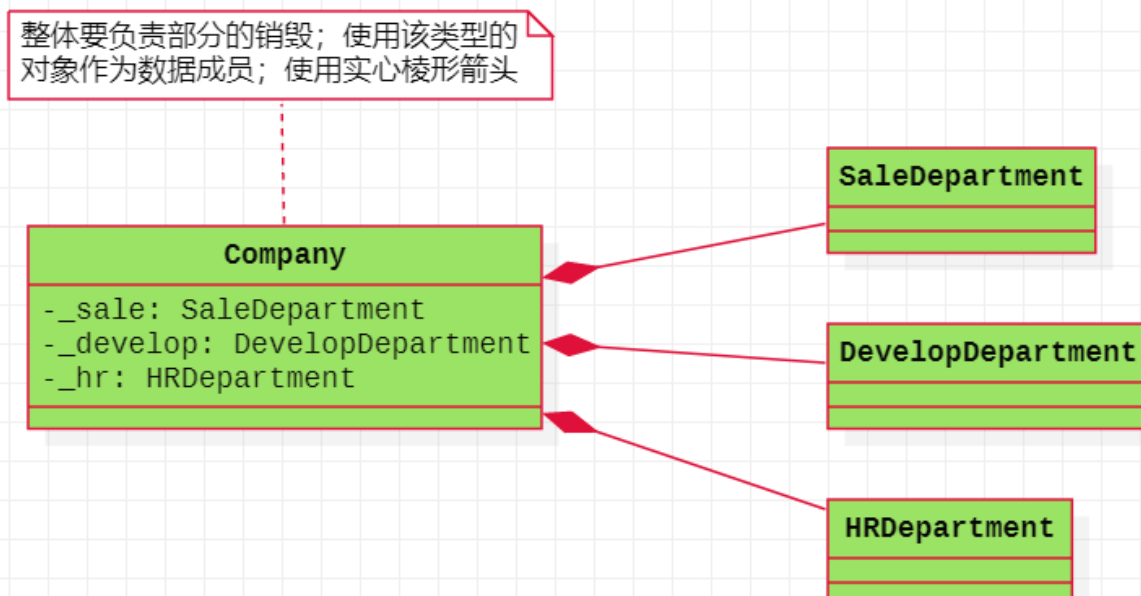
3、聚合

是一种比较强的关联关系。对象之间的关系表现为整体和局部，整体部分并不负责局部对象的销毁。语义上表现为A has B。代码上表现为：数据成员以指针或者引用形式存在。使用**空心菱形箭头**表示，部分指向整体。



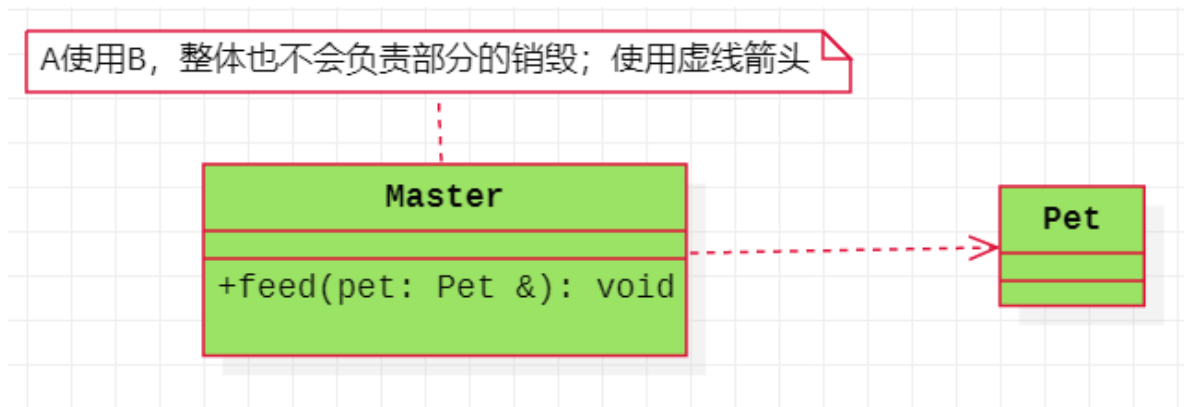
4、组合

更强的一种关联关系。对象之间的关系表现为整体和局部，整体部分负责局部对象的销毁。语义上表现为A has B。代码上表现为，数据成员以子对象成员形式存在。使用**实心菱形箭头**表示，部分指向整体。



5、依赖

是两个类之间的一种不确定的关系，语义上是一种A use B的关系，这种关系是偶然的，临时的，并非固定的。在代码上表现为：B作为A的成员**函数参数**；B作为A的成员函数的**局部变量**（B作为A的成员函数的返回值）；A的成员函数调用B的**静态方法**。使用**虚线的箭头，从A指向B**。



五种关系的总结

类与类之间的这五种关系，是类和类之间的一种普遍关系。这五种关系，从**耦合程度**来看的话是：依赖 < 关联 < 聚合 < 组合 < 继承；从**语义**上来看，继承是A is B，关联、聚合、组合是A has B，依赖是A use B的关系；从**方向**上来看，继承是类与类之间的**纵向**关系，而其他四种是类与类之间的**横向**关系。之前我们实现多态使用的是继承与虚函数这种面向对象的方法，现在知道了类与类之间的其他关系后，可以使用组合与依赖这种基于对象的方法实现。

五、面向对象的设计原则

在进行面向对象设计的时候，需要考虑类与类之间的关系，这样可以使类之间的关系更加明确。但是，除此之外，在进行面向对象设计的时候，还需要注意满足一定的设计要求，也就是面向对象的设计原则，只有遵循一定的原则，才能更好的满足软件的设计需求，更好的满足变化。而一个优良的系统设计，强调要保持**低耦合、高内聚**的要求，为了满足这个要求，面向对象的设计原则设计了七种，包括：

单一职责原则 (Single Responsibility Principle)、开闭原则 (Open Closed Principle)、里氏替换原则 (Liscov Substitution Principle)、接口分离原则 (Interface Segregation Principle)、依赖倒置原则 (Dependency Inversion Principle)、迪米特法则 (Law of Demeter -> Least Knowledge Principle)、组合复用原则 (Composite/Aggregate Reuse Principle)

1、单一职责原则

定义：一个对象应该只包含单一的职责，并且该职责被完整地封装在一个类中。单一职责原则的另一种定义方式：就一个类而言，应该仅有一个引起它变化的原因。单一职责原则是最简单的面向对象设计原则，它用于控制类的粒度大小。核心思想：一个类，最好只做一件事，只有一个引起它变化的原因。核心是：解耦与增加内聚性。

在软件系统中，一个类（大到模块，小到方法）承担的职责越多，它被复用的可能性就越小，而且一个类承担的职责过多，相当于将这些职责耦合在一起，当其中一个职责变化时可能会影响其他职责的运作，因此要将这些职责进行分离，将不同的职责封装在不同的类中，即将不同的变化原因封装在不同的类中，如果多个职责总是同时发生改变则可将它们封装在同一类中。

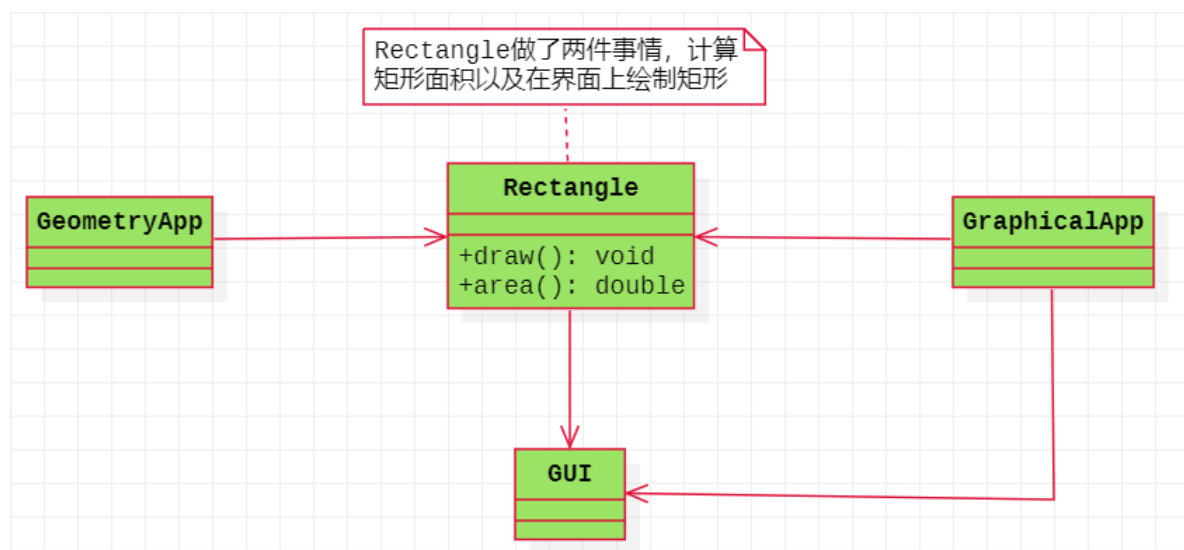
单一职责原则是实现高内聚、低耦合的指导方针，它是最简单但又最难运用的原则，需要设计人员发现类的不同职责并将其分离，而发现类的多重职责需要设计人员具有较强的分析设计能力和相关实践经验。

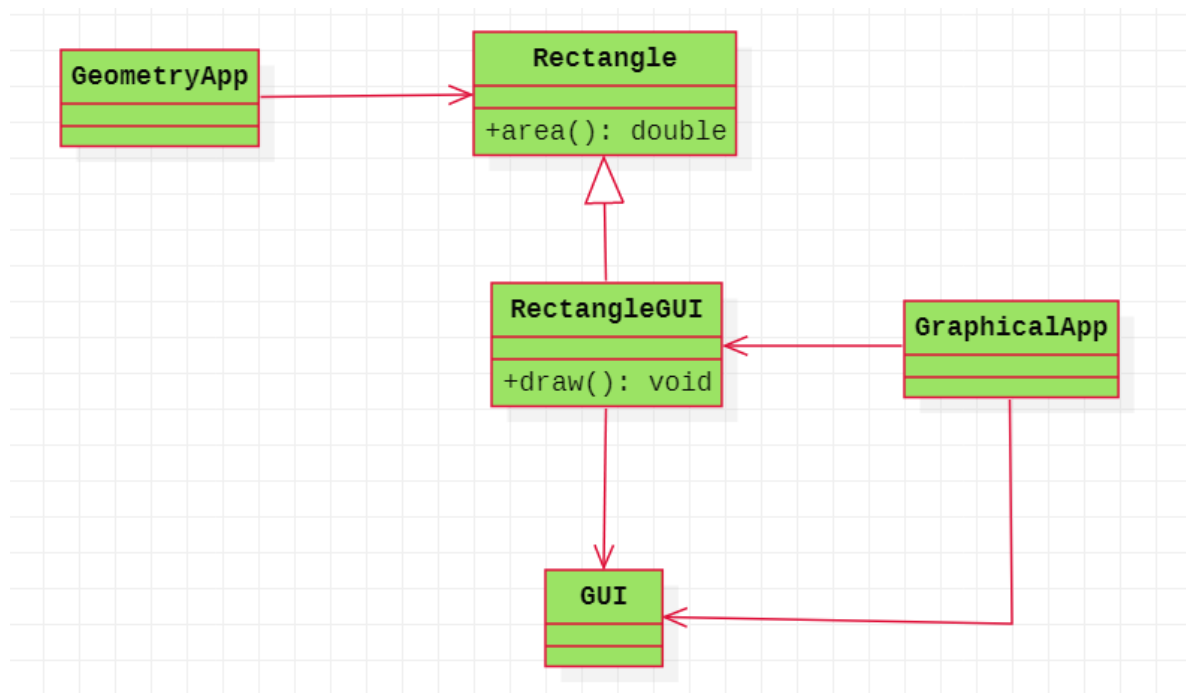


SINGLE RESPONSIBILITY PRINCIPLE

Just Because You Can, Doesn't Mean You Should

例子：将长方形类的画图与计算面积功能分开，这样的话，计算面积就是计算面积，画图就是画图。

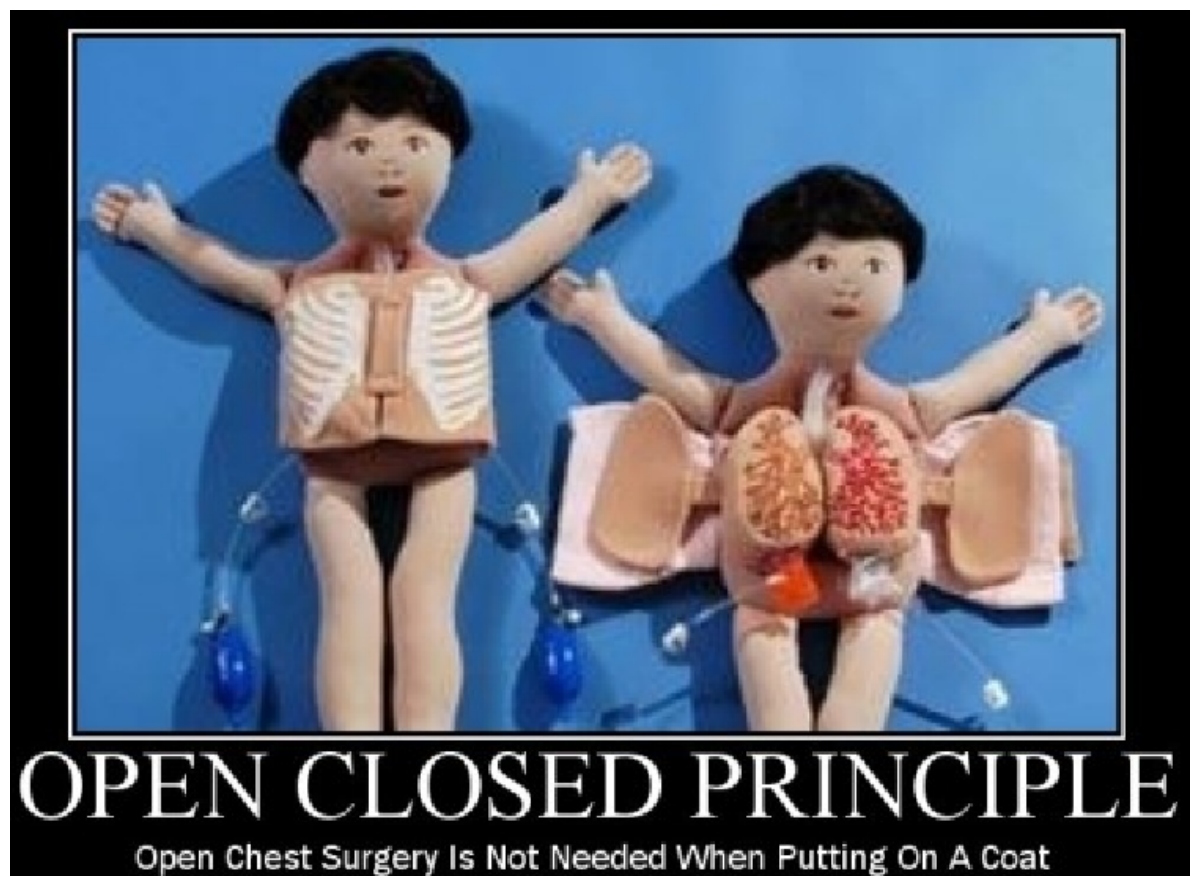




2、开闭原则

定义：软件实体应当对扩展开放，对修改关闭。开闭原则是面向对象的可复用性设计的第一块基石，它是最重要的面向对象设计原则。核心思想：对抽象编程，而不对具体编程，因为抽象相对稳定。在开闭原则的定义中，软件实体可以指一个软件模块、一个由多个类组成的局部结构或一个独立的类。开闭原则就是指软件实体应尽量在不修改原有代码的情况下进行扩展。任何软件都需要面临一个很重要的问题，即它们的需求会随时间的推移而发生变化。当软件系统需要面对新的需求时应该尽量保证系统的设计框架是稳定的。如果一个软件设计符合开闭原则，那么可以非常方便地对系统进行扩展，而且在扩展时无须修改现有代码，使得软件系统在拥有适应性和灵活性的同时具备较好的稳定性和延续性。随着软件规模越来越大，软件寿命越来越长，软件维护成本越来越高，设计满足开闭原则的软件系统也变得越来越重要。

为了满足开闭原则，需要对系统进行抽象化设计，抽象化是开闭原则的关键。在C++中，可以为系统定义一个相对稳定的抽象层，而将不同的实现行为移至具体的实现层中完成。在C++语言中提供了抽象类的机制，可以通过它定义系统的抽象层，再通过具体类来进行扩展。如果需要修改系统的行为，无须对抽象层进行任何改动，只需要增加新的具体类来实现新的业务功能即可，实现在不修改已有代码的基础上扩展系统的功能，达到开闭原则的要求。



例子：版本升级，不改源代码，原来的功能就是可以使用的。计算器代码的实现：分别设计加减乘除，不如直接使用继承，实现虚方法。

计算器类，该类包含四个方法

Calculator

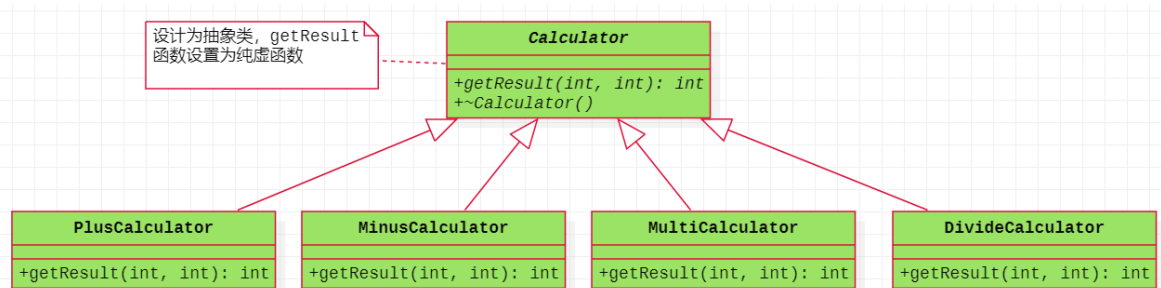
```
+plus(int, int): int  
+minus(int, int): int  
+multiply(int, int): int  
+divide(int, int): int
```

```
1 class Calculator  
2 {  
3     public:  
4         int plus(int x, int y)  
5         {  
6             return x + y;  
7         }  
8  
9         int minus(int x, int y)  
10        {  
11            return x - y;  
12        }
```

```

13
14     int multiply(int x, int y)
15     {
16         return x * y;
17     }
18
19     int divide(int x, int y)
20     {
21         if(0 != y)
22         {
23             return x/y;
24         }
25         else
26         {
27             cout << "除数不能为0" << endl;
28             return 1 << 31;
29         }
30     }
31 };

```



```

1  class calculator
2  {
3  public:
4      virtual int getResult(int x, int y) = 0;
5      virtual ~calculator() {}
6  };
7
8  class PlusCalculator
9  : public calculator
10 {
11 public:
12     virtual int getResult(int x, int y)
13     {
14         return x + y;
15     }
16
17     ~PlusCalculator()
18     {
19     }
20 };
21
22 class MinusCalculator
23 : public calculator
24 {
25 public:
26     virtual int getResult(int x, int y)

```

```

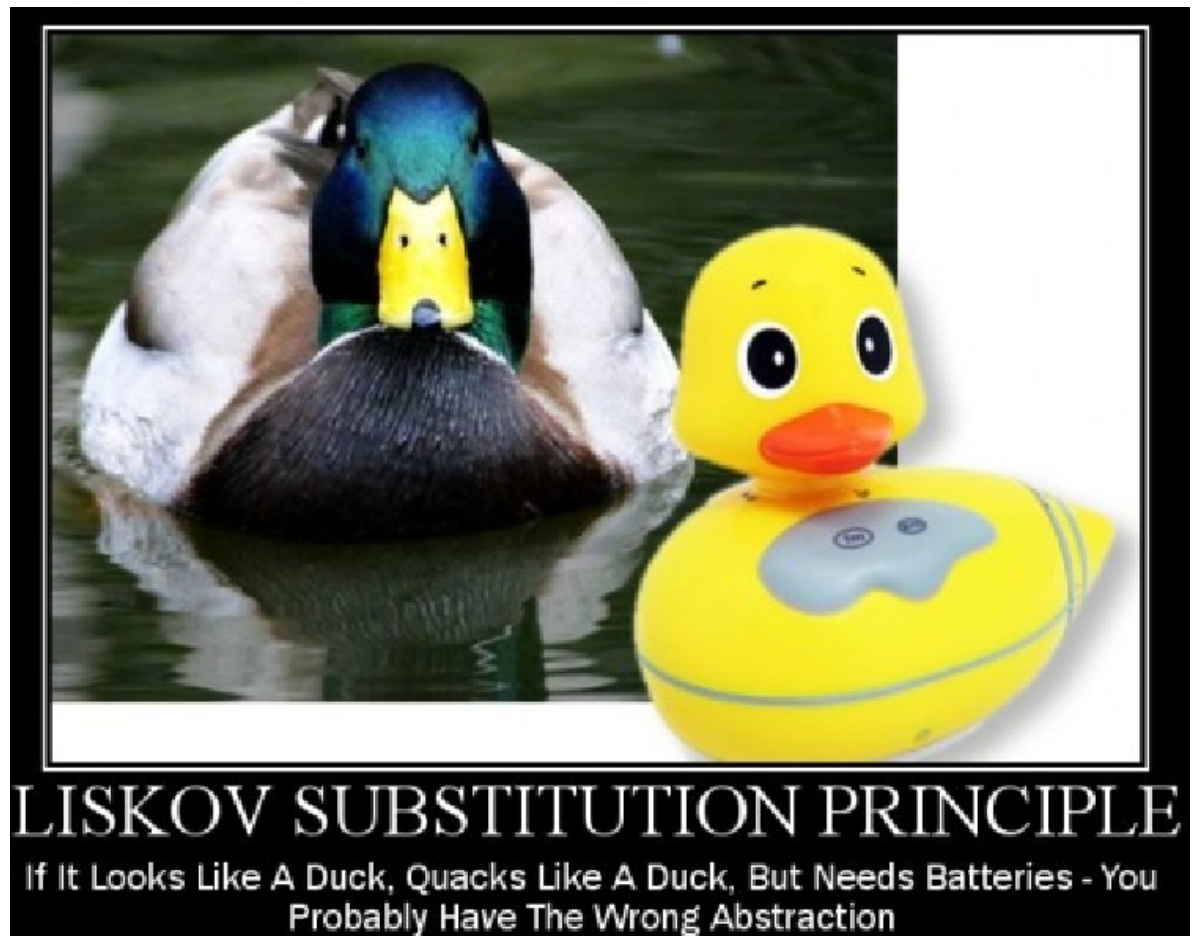
27     {
28         return x - y;
29     }
30
31     ~MinusCalculator()
32     {
33     }
34 };
35 class MultiplyCalculator
36 : public Calculator
37 {
38 public:
39     virtual int getResult(int x, int y)
40     {
41         return x * y;
42     }
43
44     ~MultiplyCalculator()
45     {
46     }
47 };
48
49 class DivideCalculator
50 : public Calculator
51 {
52 public:
53     virtual int getResult(int x, int y)
54     {
55         if(0 != y)
56         {
57             return x/y;
58         }
59         else
60         {
61             cout << "除数不能为0" << endl;
62             return 1 << 31;
63         }
64     }
65
66     ~DivideCalculator()
67     {
68     }
69 };

```

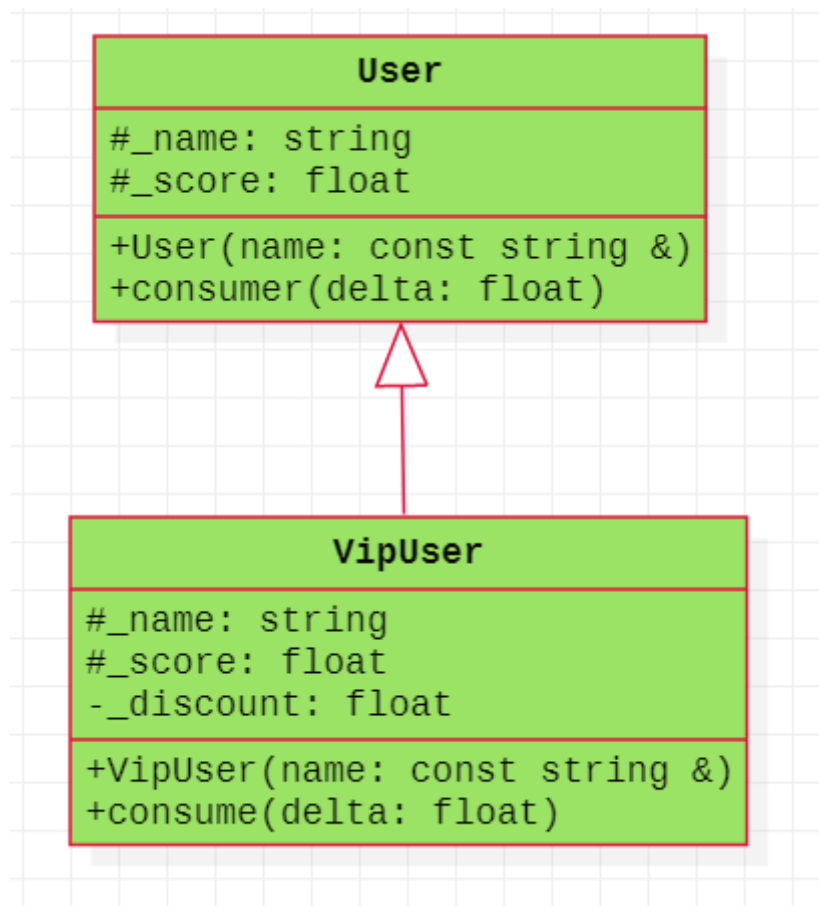
3、里氏替换原则

定义：所有引用基类（父类）的地方必须能透明地使用其派生类（子类）的对象。里氏代换原则表明，在软件中将一个基类对象替换成它的派生类对象，程序将不会产生任何错误和异常，反过来则不成立，如果一个软件实体使用的是一个派生类对象，那么它不一定能够使用基类对象。核心思想：派生类必须能够替换其基类。派生类可以扩展基类的功能，但不能改变基类原有的功能。表现为：派生类（子类）可以实现基类（父类）的抽象方法，表现多态，但不能覆盖基类（父类）的非抽象方法，当然派生类（子类）可以有自己独特的个性。例如我喜欢动物，那我一定喜欢狗，因为狗是动物的派生类（子类）；但是我喜欢狗，不能据此断定我喜欢所有的动物。里氏代换原则是实现开闭原则的重要方式之一，由于在使用基类（父类）对象的地方都可以使用派生类（子类）对象，因此在程序中尽量使用基类类型来对对象进行定义，而在运行时再确定其派生类（子类）类型，用派生类（子类）对象来替换基类（父类）

对象。在运用里氏代换原则时应该将基类（父类）设计为抽象类或者接口，让派生类（子类）继承基类（父类）或实现父接口，并实现在基类（父类）中声明的方法，在运行时派生类（子类）实例替换基类（父类）实例，可以很方便地扩展系统的功能，无须修改原有派生类（子类）的代码，增加新的功能可以通过增加一个新的派生类（子类）来实现。



有代码的实现：（注意接口的概念，功能服务，普通函数、自由函数，类的public成员函数，虚函数，纯虚函数），注意与隐藏做区分，派生类要保有原来的功能，然后在此基础上扩展该功能。

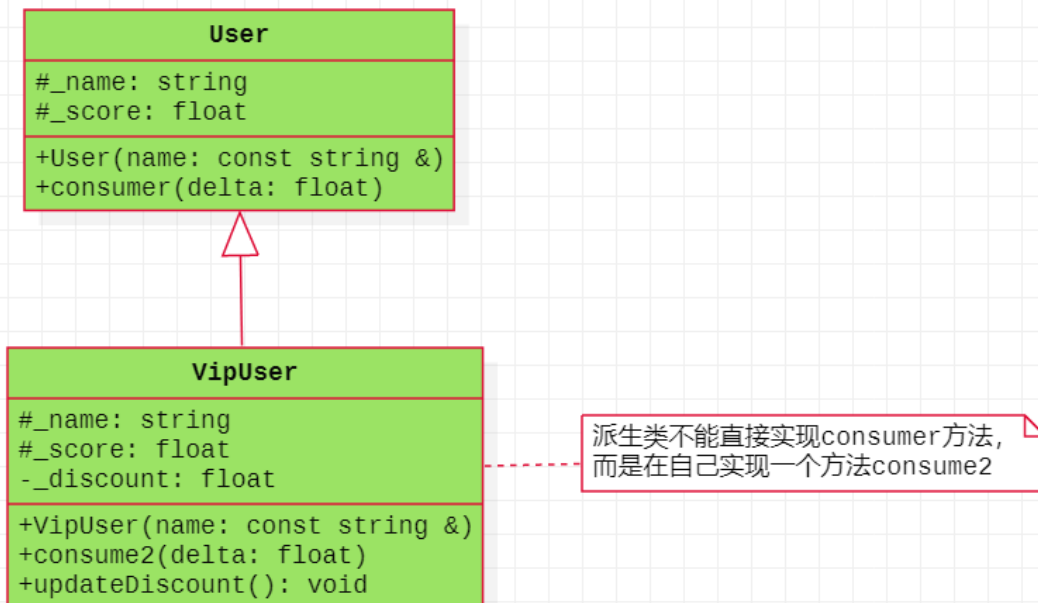


```
1  class User
2  {
3  public:
4      User(const string &name)
5          : _name(name)
6          , _score(0)
7      {
8      }
9
10     void consume(float delta)
11     {
12         cout << "User::consume()" << endl;
13         _score += delta;
14         cout << ">> " << _name << " consume " << delta << endl;
15     }
16
17 protected:
18     string _name;
19     float _score;
20 };
21
22 class VipUser
23 : public User
24 {
25 public:
26     vipUser(const string &name)
27         : User(name)
28         , _discount(1)
29     {
30     }
31 }
```

```

32 void consume(float delta)
33 {
34     cout << "vipUser::consume()" << endl;
35
36     float tmp = delta * _discount;
37     _score += tmp;
38
39     if(_score > 1000 && _score < 5000)
40     {
41         _discount = 0.9;
42     }
43
44     cout << ">> " << _name << " consume " << tmp << endl;
45 }
46 private:
47     float _discount;
48 };
49
50 void test()
51 {
52     User user("刘德华");
53     user.consume(2000);
54     user.consume(2000);
55
56     cout << endl;
57     vipUser vip("张学友");
58     vip.consume(2000);
59     vip.consume(2000); //改变了基类的方法，隐藏
60 }

```



```

1 class User
2 {
3 public:
4     User(const string &name)
5         : _name(name)
6         , _score(0)

```

```

7      {
8      }
9
10     void consume(float delta)
11     {
12         _score += delta;
13
14         cout << ">> " << _name << " consume " << delta << endl;
15     }
16
17 protected:
18     string _name;
19     float _score;
20 };
21
22 class VipUser
23 : public User
24 {
25 public:
26     VipUser(const string &name)
27         : User(name)
28         , _discount(1)
29     {
30     }
31
32     void consume2(float delta)
33     {
34         float realDelta = delta * _discount;
35         _score += realDelta;
36         updateDiscount();
37         cout << ">> " << _name << " consume " << realDelta << endl;
38     }
39
40     void updateDiscount()
41     {
42         if(_score > 10000)
43         {
44             _discount = 0.7;
45         }
46         else if(_score > 5000)
47         {
48             _discount = 0.8;
49         }
50         else if(_score > 1000)
51         {
52             _discount = 0.9;
53         }
54         else
55         {
56             _discount = 1;
57         }
58     }
59 private:
60     float _discount;
61 };
62
63 void test()
64 {

```

```

65     User user("lili");
66     user.consume(2000);
67     user.consume(2000);
68
69     cout << endl;
70     vipUser vip("lucy");
71     vip.consume2(2000);
72     vip.consume2(20000); //添加自己的个性
73     vip.consume(20000); //基类的方法还是保留在,
74
75 }

```

4、接口分离原则

定义：客户端不应该依赖那些它不需要的接口。核心思想：使用多个小的专门的接口，而不要使用一个大的总接口。

根据接口隔离原则，当一个接口太大时需要将它分割成一些更细小的接口，使用该接口的客户端仅需知道与之相关的方法即可。每一个接口应该承担一种相对独立的角色，不干不该干的事，该干的事都要干。这里的“接口”往往有两种不同的含义：一种是指一个类型所具有的方法特征的集合，仅仅是一种逻辑上的抽象；另外一种是指C++语言中声明纯虚函数的抽象类。对于这两种不同的含义，接口隔离原则的表达方式以及含义也有所不同。（1）当把“接口”理解成一个类型所提供的所有方法特征的集合的时候，这就是一种逻辑上的概念，接口的划分将直接带来类型的划分。可以把接口理解成角色，一个接口只能代表一个角色，每个角色都有它特定的一个接口，此时这个原则可以叫“角色隔离原则”。

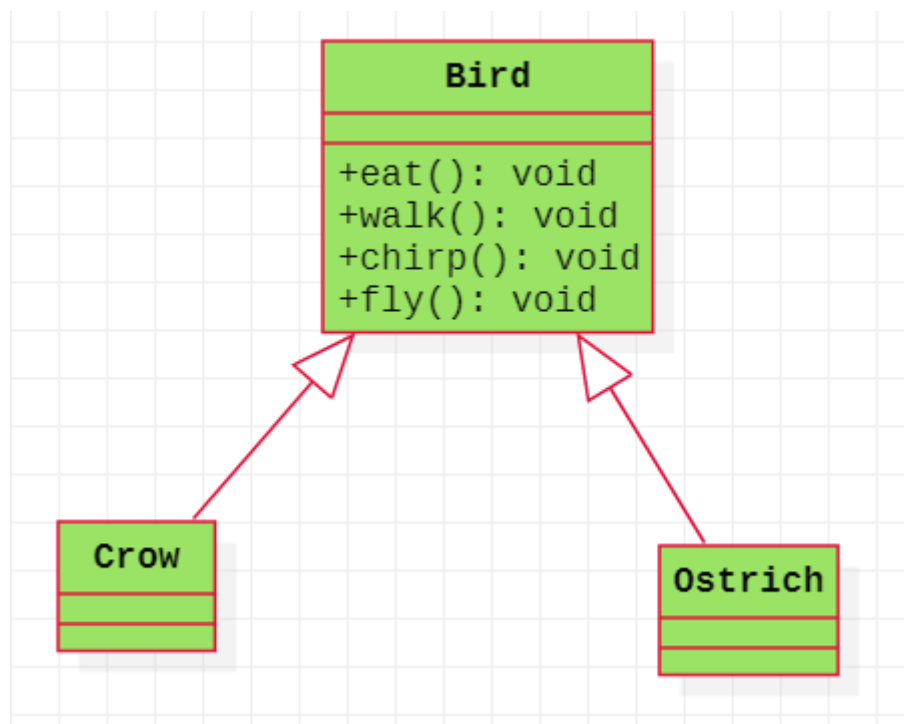
（2）如果把“接口”理解成狭义的C++语言的抽象类，那么接口隔离原则表达的意思是指接口仅提供客户端需要的行为，客户端不需要的行为则隐藏起来，应当为客户端提供尽可能小的单独的接口，而不要提供大的总接口。在C++语言中，实现一个接口需要实现该接口中定义的所有方法，因此大的总接口使用起来不一定很方便，为了使接口的职责单一，需要将大接口中的方法根据其职责不同分别放在不同的小接口中，以确保每个接口使用起来都较为方便，并都承担某一单一角色。接口应该尽量细化，同时接口中的方法应该尽量少，每个接口只包含一个客户端（如子模块或业务逻辑类）所需的方法即可，这种机制也称为“定制服务”，即为不同的客户端提供宽窄不同的接口。



INTERFACE SEGREGATION PRINCIPLE

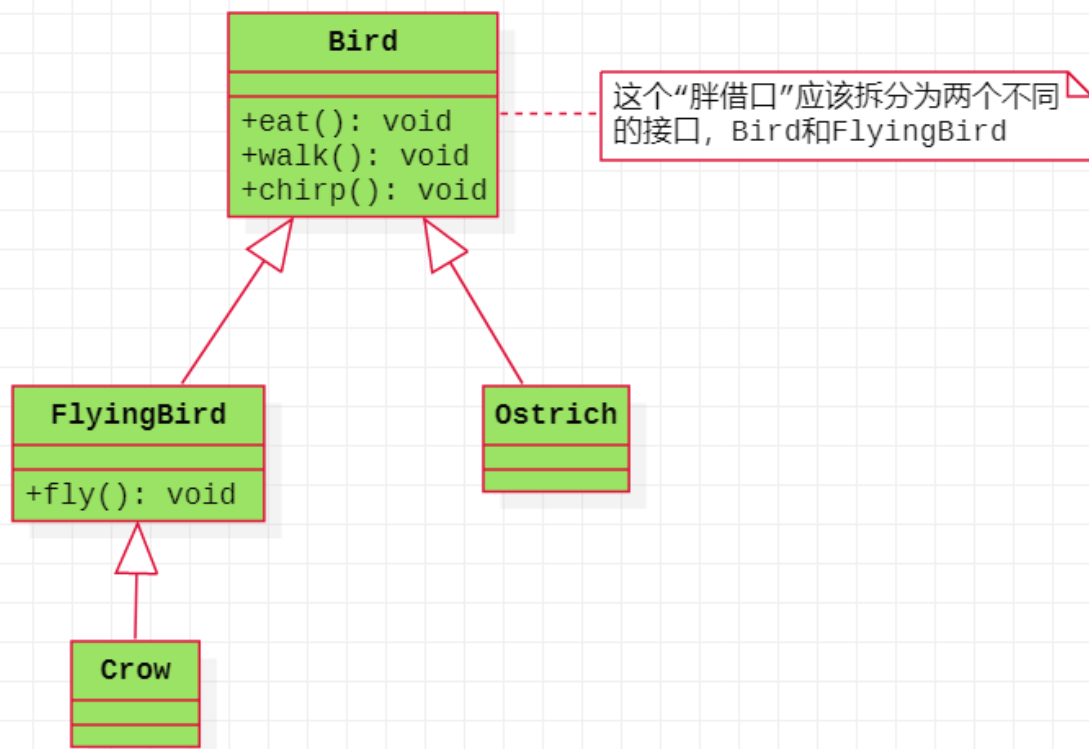
You Want Me To Plug This In, Where?

例子：鸟与鸵鸟



```
1 class Bird
2 {
3     public:
4         virtual void eat() = 0;
5         virtual void walk() = 0;
6         virtual void chirp() = 0;
7         virtual void fly() = 0;
8         virtual ~Bird() {}
9 };
10
```

```
11 class Crow
12 : public Bird
13 {
14 public:
15     void eat() override
16     {
17         cout << "乌鸦喝水" << endl;
18     }
19
20     void walk() override
21     {
22         cout << "乌鸦散步" << endl;
23     }
24
25     void chirp() override
26     {
27         cout << "乌鸦蹦蹦跳跳" << endl;
28     }
29
30     void fly() override
31     {
32         cout << "乌鸦飞翔" << endl;
33     }
34 };
35
36 class Ostrich
37 : public Bird
38 {
39 public:
40     void eat() override
41     {
42         cout << "鸵鸟吃东西" << endl;
43     }
44
45     void walk() override
46     {
47         cout << "鸵鸟散步" << endl;
48     }
49
50     void chirp() override
51     {
52         cout << "鸵鸟蹦蹦跳跳" << endl;
53     }
54
55     void fly() override
56     {
57         cout << "鸵鸟不会飞翔" << endl;
58     }
59 };
```



```
1  class Bird
2  {
3  public:
4      virtual void eat() = 0;
5      virtual void walk() = 0;
6      virtual void chirp() = 0;
7      virtual ~Bird() {}
8  };
9
10 class FlyingBird
11 : public Bird
12 {
13 public:
14     virtual void fly() = 0;
15     ~FlyingBird() {}
16 };
17
18 class Crow
19 : public FlyingBird
20 {
21 public:
22     void eat() override
23     {
24         cout << "乌鸦喝水" << endl;
25     }
26
27     void walk() override
28     {
29         cout << "乌鸦散步" << endl;
30     }
31
32     void chirp() override
33     {
34         cout << "乌鸦蹦蹦跳跳" << endl;
35     }
36 }
```

```

36
37     void fly() override
38     {
39         cout << "乌鸦飞翔" << endl;
40     }
41 };
42
43 class Ostrich
44 : public Bird
45 {
46 public:
47     void eat() override
48     {
49         cout << "鸵鸟吃东西" << endl;
50     }
51
52     void walk() override
53     {
54         cout << "鸵鸟散步" << endl;
55     }
56
57     void chirp() override
58     {
59         cout << "鸵鸟蹦蹦跳跳" << endl;
60     }
61 };

```

5、依赖倒置原则

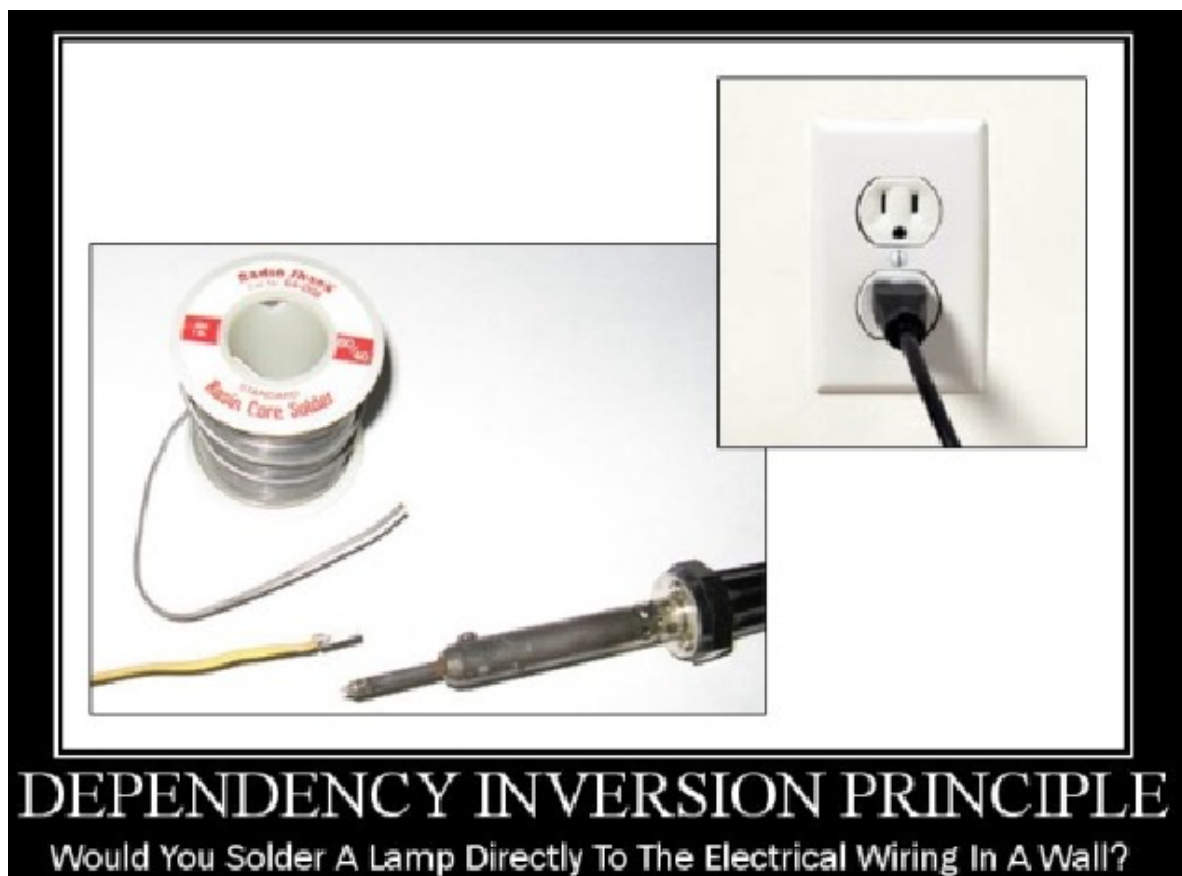
定义：高层模块不应该依赖低层模块，它们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。简单来说，依赖倒置原则要求针对接口编程，不要针对实现编程。核心思想：面向接口编程，依赖于抽象。

如果说开闭原则是面向对象设计的目标，那么依赖倒置原则就是面向对象设计的主要实现机制之一。依赖倒置原则要求在程序代码中传递参数时或在关联关系中尽量引用层次高的抽象层类，即使用接口和抽象类进行变量类型声明、参数类型声明、方法返回类型声明，以及数据类型的转换等，而不要用具体类来做这些事情。为了确保该原则的应用，一个具体类应当只实现接口或抽象类中声明过的方法，而不要给出多余的方法，否则将无法调用到在子类中增加的新方法。

在引入抽象层后，系统将具有很好的灵活性，在程序中尽量使用抽象层进行编程，而将具体类写在配置文件中，这样如果系统行为发生变化，只需要对抽象层进行扩展，并修改配置文件，而无须修改原有系统的源代码，在不修改的情况下扩展系统的功能，满足开闭原则的要求。

在实现依赖倒置原则时需要针对抽象层编程，而将具体类的对象通过依赖注入（Dependence Injection, DI）的方式注入到其他对象中，依赖注入是指当一个对象要与其他对象发生依赖关系时采用抽象的形式来注入所依赖的对象。常用的注入方式有3种，分别是构造注入、设值注入和接口注入。构造注入是指通过构造函数来传入具体类的对象，设值注入是指通过Setter方法来传入具体类的对象，而接口注入是指通过在接口中声明的业务方法来传入具体类的对象。这些方法在定义时使用的是抽象类型，在运行时再传入具体类型的对象，由子类对象来覆盖父类对象。

在大多数情况下，开闭原则、里氏替换原则和依赖倒置原则会同时出现，开闭原则是目标，里氏替换原则是基础，依赖倒置原则是手段。



例子：银行业务类。

该类承担了所有的具体业务，如果具体业务有变化，该类就要被修改，因为具体的是容易变化的

BankWorker

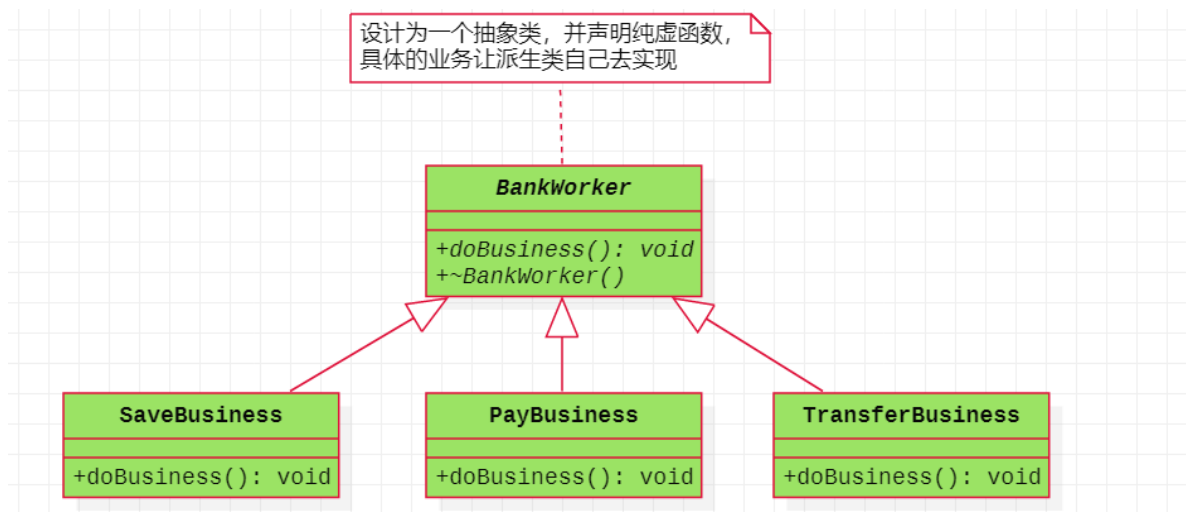
+saveService(): void
+payService(): void
+transferService(): void

```
1 class BankWorker
2 {
3 public:
4     void saveService()
5     {
6         cout << "办理存款业务" << endl;
7     }
8
9     void payService()
10    {
11        cout << "办理支付业务" << endl;
12    }
13
```

```

14     void transfersService()
15     {
16         cout << "办理转账业务" << endl;
17     }
18
19 };
20
21 void doSaveBusiness(BankWorker *worker)
22 {
23     worker->saveService();
24 }
25
26 void doPayBusiness(BankWorker *worker)
27 {
28     worker->payService();
29 }
30
31 void doTransferBusiness(BankWorker *worker)
32 {
33     worker->transfersService();
34 }
35
36 void test()
37 {
38     unique_ptr<BankWorker> worker(new BankWorker());
39     doSaveBusiness(worker.get());
40     doPayBusiness(worker.get());
41     doTransferBusiness(worker.get());
42 }

```



```

1  class BankWorker
2  {
3  public:
4      virtual void doBusiness() = 0;
5      virtual ~BankWorker() {}
6  };
7
8  class SaveBankWorker
9  : public BankWorker
10 {

```

```

11 public:
12     void doBusiness() override
13     {
14         cout << "办理存款业务" << endl;
15     }
16 };
17
18 class PayBankWorker
19 : public BankWorker
20 {
21 public:
22     void doBusiness() override
23     {
24         cout << "办理支付业务" << endl;
25     }
26 };
27
28 class TransferBankWorker
29 : public BankWorker
30 {
31 public:
32     void doBusiness() override
33     {
34         cout << "办理转账业务" << endl;
35     }
36 };
37
38 void doBusiness(BankWorker *worker)
39 {
40     worker->doBusiness();
41 }
42
43 void test()
44 {
45     unique_ptr<BankWorker> saveworker(new SaveBankWorker());
46     unique_ptr<BankWorker> payworker(new PayBankWorker());
47     unique_ptr<BankWorker> transferworker(new TransferBankWorker());
48
49     doBusiness(saveworker.get());
50     doBusiness(payworker.get());
51     doBusiness(transferworker.get());
52 }

```

6、迪米特法则

定义：每一个软件单位对其他单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。

迪米特法则要求一个软件实体应当尽可能少地与其他实体发生相互作用。如果一个系统符合迪米特法则，那么当其中的某一个模块发生修改时就会尽量少地影响其他模块，扩展会相对容易，这是对软件实体之间通信的限制，迪米特法则要求限制软件实体之间通信的宽度和深度。应用迪米特法则可降低系统的耦合度，使类与类之间保持松散的耦合关系。当两个类之间直接通信的时候，会造成高度依赖的后果（高耦合）。解决此问题的办法，尽量避免两个类直接接触（低耦合），通过一个第三者做转发。

在迪米特法则中，对于一个对象，其朋友包括以下几个类：

- (1) 当前对象本身（this）。
- (2) 以参数形式传入到当前对象方法中的对象。

- (3) 当前对象的成员对象。
- (4) 如果当前对象的成员对象是一个集合，那么集合中的元素也都是朋友。
- (5) 当前对象所创建的对象。

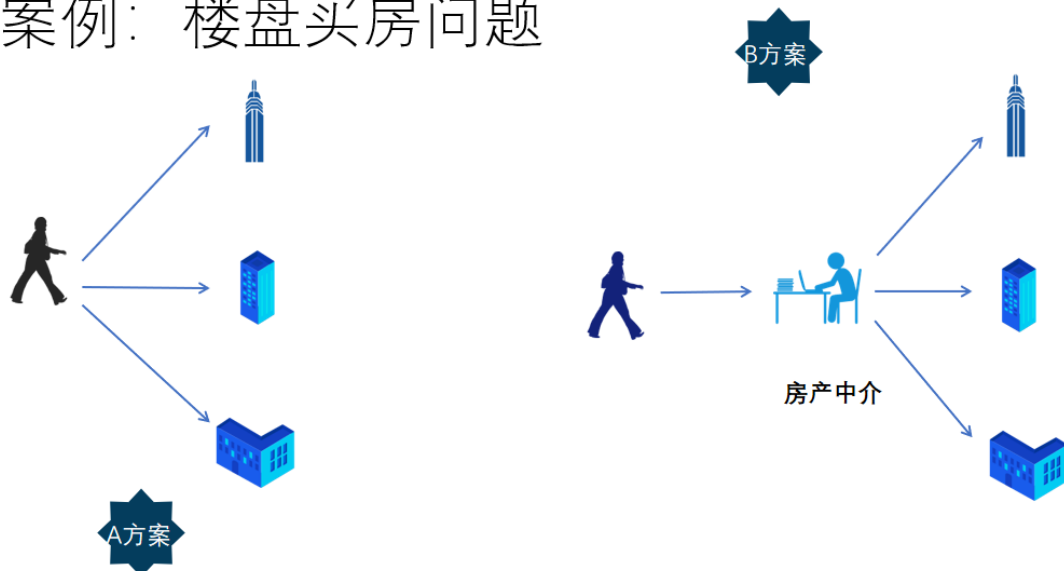
任何一个对象如果满足上面的条件之一，就是当前对象的“朋友”，否则就是“陌生人”。在应用迪米特法则时，一个对象只能与直接朋友发生交互，不要与“陌生人”发生直接交互，这样做可以降低系统的耦合度，一个对象的改变不会给太多其他对象带来影响。

迪米特法则要求在设计系统时应该尽量减少对象之间的交互，如果两个对象之间不必彼此直接通信，那么这两个对象就不应当发生任何直接的相互作用，如果其中一个对象需要调用另一个对象的方法，可以通过“第三者”转发这个调用。简而言之，就是通过引入一个合理的“第三者”来降低现有对象之间的耦合度。

在将迪米特法则运用到系统设计时要注意下面几点：在类的划分上应当尽量创建松耦合的类，类之间的耦合度越低，就越有利于复用，一个处在松耦合中的类一旦被修改不会对关联的类造成太大影响；在类的结构设计上，每一个类都应当尽量降低其成员变量和成员函数的访问权限；在类的设计上，只要有可能，一个类型应当设计成不变类；在对其他类的引用上，一个对象对其他对象的引用应当降到最低。

例如：使用买房者购房的一个例子。

案例：楼盘买房问题



A方案，购房者需要挨个楼盘的去知道确切的消息，需要对每个楼盘都有详细的了解，而B方案是购房者可以不用知道每个楼盘的具体信息，直接找个中介，告诉中介自己的需求，让中介进行推荐，这样购房者与楼盘之间的关系就不是那么强相关。

方案A的代码实现

```
1  class Building
2  {
3  public:
4      virtual void sale() = 0;
5      virtual string getQuality() = 0;
6      virtual ~Building(){}
7  };
8
9  class wankeBuilding
10 : public Building
11 {
12 public:
13     wankeBuilding()
14         : _quality("高品质")
15     {
```

```

16     }
17
18
19     void sale() override
20     {
21         cout << "万科楼盘" << _quality << "的房子被出售" << endl;
22     }
23
24     string getQuality() override
25     {
26         return _quality;
27     }
28 private:
29     string _quality;
30 };
31
32 class HengdaBuilding
33 : public Building
34 {
35 public:
36     HengdaBuilding()
37     : _quality("低品质")
38     {
39     }
40
41     void sale() override
42     {
43         cout << "恒大楼盘" << _quality << "的房子被出售" << endl;
44     }
45
46     string getQuality() override
47     {
48         return _quality;
49     }
50 private:
51     string _quality;
52 };
53
54 void test()
55 {
56     Building *pbuidingA = new WankeBuilding();
57     Building *pbuidingB = new HengdaBuilding();
58
59     string demand = "低品质";
60
61     if(pbuidingA->getQuality() == demand)
62     {
63         pbuidingA->sale();
64     }
65
66     if(pbuidingB->getQuality() == demand)
67     {
68         pbuidingB->sale();
69     }
70 }

```

方案B的代码实现

```

1  class Mediator
2  {
3  public:
4      Mediator()
5      {
6          Building *pbuildingA = new WankeBuilding();
7          Building *pbuildingB = new HengdaBuilding();
8          _buildings.push_back(pbuildingA);
9          _buildings.push_back(pbuildingB);
10     }
11
12     Building *findBuilding(const string &quality)
13     {
14         for(auto &building : _buildings)
15         {
16             if(building->getQuality() == quality)
17             {
18                 return building;
19             }
20         }
21
22         return nullptr;
23     }
24
25     ~Mediator()
26     {
27         for(auto &building : _buildings)
28         {
29             if(building)
30             {
31                 delete building;
32             }
33         }
34     }
35 private:
36     vector<Building *> _buildings;
37 };
38
39 void test()
40 {
41     string demand = "低品质";
42     Mediator mediator;
43
44     Building *pbuilding = mediator.findBuilding(demand);
45     if(pbuilding)
46     {
47         pbuilding->sale();
48     }
49     else
50     {
51         cout << "没有符合要求的楼盘" << endl;
52     }
53 }

```

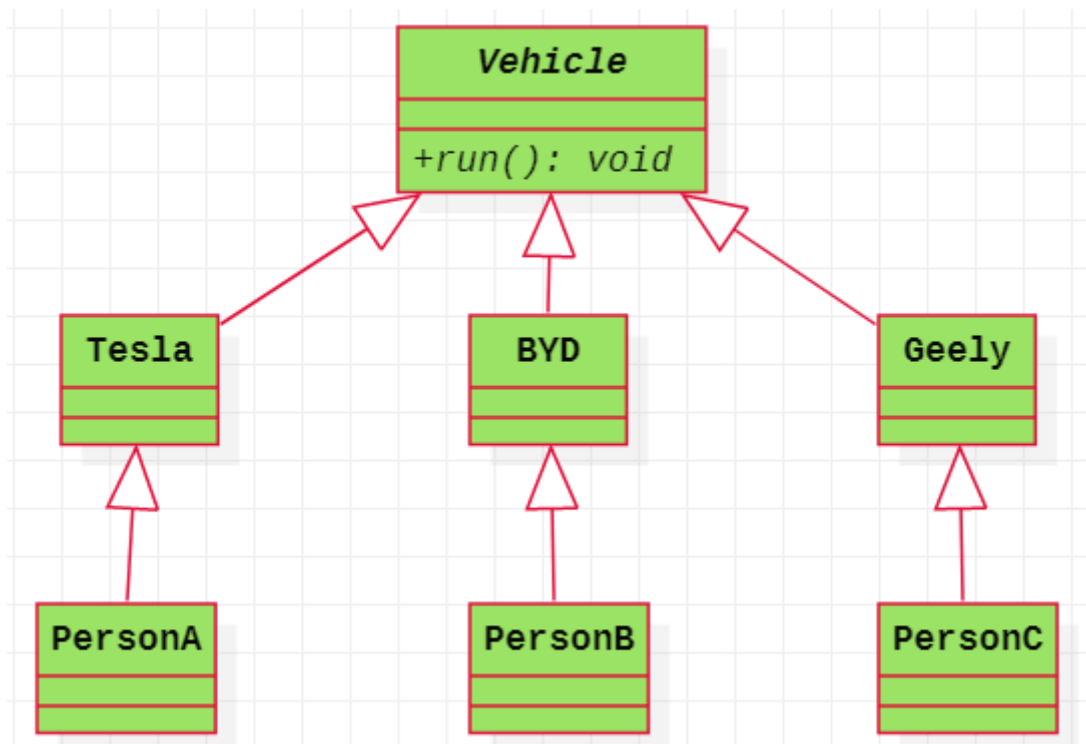
7、组合复用原则

定义：优先使用对象组合，而不是通过继承来达到复用的目的。合成复用原则就是在一个新的对象里通过关联关系（包括组合关系和聚合关系）来使用一些已有的对象，使之成为新对象的一部分，新对象通过委派调用已有对象的方法达到复用功能的目的。简而言之，在复用时尽量使用组合/聚合关系（关联关系），少用继承。

在面向对象设计中可以通过两种方法在不同的环境中复用已有的设计和实现，即通过组合/聚合关系或通过继承，但首先应该考虑使用组合/聚合，组合/聚合可以使系统更加灵活，降低类与类之间的耦合度，一个类的变化对其他类造成的影响相对较少；其次才考虑继承，在使用继承时需要严格遵循里氏代换原则，有效使用继承会有助于对问题的理解，降低复杂度，而滥用继承反而会增加系统构建和维护的难度以及系统的复杂度，因此需要慎重使用继承复用。

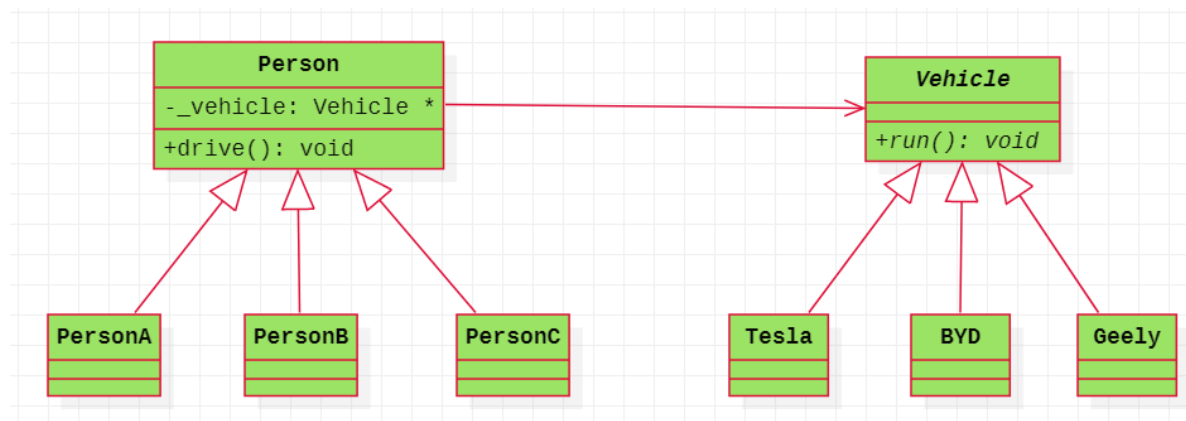
通过继承来进行复用的主要问题在于继承复用会破坏系统的封装性，因为继承会将基类的实现细节暴露给子类，由于基类的某些内部细节对子类来说是可见的，所以这种复用又称“白箱”复用，如果基类发生改变，那么子类的实现也不得不发生改变；从基类继承而来的实现是静态的，不可能在运行时发生改变，没有足够的灵活性；而且继承只能在有限的环境中使用（如类没有声明为不能被继承）。

由于组合/聚合关系可以将已有的对象纳入到新对象中，使之成为新对象的一部分，因此新对象可以调用已有对象的功能，这样做可以使成员对象的内部实现细节对于新对象不可见，所以这种复用又称为“黑箱”复用，相对继承关系而言，其耦合度相对较低，成员对象的变化对新对象的影响不大，可以在新对象中根据实际需要有针对性地调用成员对象的操作；合成复用可以在运行时动态进行，新对象可以动态地引用与成员对象类型相同的其他对象。一般而言，如果两个类之间是“Has-A”的关系应使用组合/聚合，如果是“Is-A”的关系可以使用继承。“Is-A”是严格的分类学意义上的定义，意思是一个类是另一个类的“一种”；而“Has-A”则不同，它表示某一个角色具有某一项责任。



```
1  class vehicle
2  {
3  public:
4      virtual void run() = 0;
5      virtual ~vehicle() {}
6  };
7
8  class Tesla
9  : public vehicle
10 {
11 public:
12     void run() override
13     {
```

```
14         cout << "Model Y start..." << endl;
15     }
16 };
17
18 class BYD
19 : public Vehicle
20 {
21 public:
22     void run() override
23     {
24         cout << "汉EV start..." << endl;
25     }
26 };
27
28 class Geely
29 : public Vehicle
30 {
31 public:
32     void run() override
33     {
34         cout << "Geely LYNK03 start..." << endl;
35     }
36 };
37
38 class PersonA
39 : public Tesla
40 {
41
42 };
43
44 class PersonB
45 : public BYD
46 {
47
48 };
49
50 class PersonC
51 : public Geely
52 {
53
54 };
55
56 void test()
57 {
58     PersonA pa;
59     pa.run();
60
61     PersonB pb;
62     pb.run();
63
64     PersonC pc;
65     pc.run();
66 }
```



```

1  class Vehicle
2  {
3  public:
4      virtual void run() = 0;
5      virtual ~Vehicle() {}
6  };
7
8  class Tesla
9  : public Vehicle
10 {
11 public:
12     void run() override
13     {
14         cout << "Model Y start..." << endl;
15     }
16 };
17
18 class BYD
19 : public Vehicle
20 {
21 public:
22     void run() override
23     {
24         cout << "汉EV start..." << endl;
25     }
26 };
27
28 class Geely
29 : public Vehicle
30 {
31 public:
32     void run() override
33     {
34         cout << "Geely LYNK03 start..." << endl;
35     }
36 };
37
38 class Person
39 {
40 public:
41     void getVehicle(Vehicle *vehicle)
42     {
43         _vehicle = vehicle;
44     }
45

```

```
46     void drive()
47     {
48         _vehicle->run();
49     }
50 private:
51     vehicle *_vehicle;
52 };
53
54 void test()
55 {
56     Person person;
57     unique_ptr<Vehicle> tesla(new Tesla());
58     unique_ptr<Vehicle> han(new BYD());
59     unique_ptr<Vehicle> geely(new Geely());
60
61     person.getVehicle(tesla.get());
62     person.drive();
63
64     person.getVehicle(han.get());
65     person.drive();
66
67     person.getVehicle(geely.get());
68     person.drive();
69 }
```

七大设计原则的总结

原则的目的只有一个：降低对象之间的耦合，增加程序的可复用性、可扩展性和可维护性。

设计原则名称	设计原则简介	重要性
单一职责原则	类的职责要单一，不能将太多的职责放在一个类中	四颗星
开闭原则	软件实体对扩展是开放的，但对修改是关闭的，即在不修改一个软件的基础上去扩展其功能	五颗星
里氏替换原则	在软件系统中，一个可以接受基类对象的地方必然可以接收一个派生类对象	四颗星
依赖倒置原则	要针对抽象编程，而不是针对具体编程	五颗星
接口隔离原则	使用多个专门的接口来取代一个统一的接口	二颗星
迪米特法则	一个软件实体对其他实体的引用越少越好，或者说如果两个类彼此直接通信，那么这两个类就不应当发生直接相互作用，而是通过引入一个第三者发生间接交互	三颗星
合成复用原则	在系统中应该尽量多使用组合和聚合关联关系，尽量少使用甚至不使用继承关系	四颗星

六、设计模式

1、概述

设计模式的由来，1977 年有位美国著名建筑大师提出，他叫Christopher Alexander（克里斯托弗·亚历山大）。他拥有剑桥大学数学硕士学位和建筑学学士学位，以及哈佛大学建筑学博士学位，有“模式它爹”的称号。1990 年，软件工程界开始研讨设计模式的话题。1994 年，艾瑞克·伽马

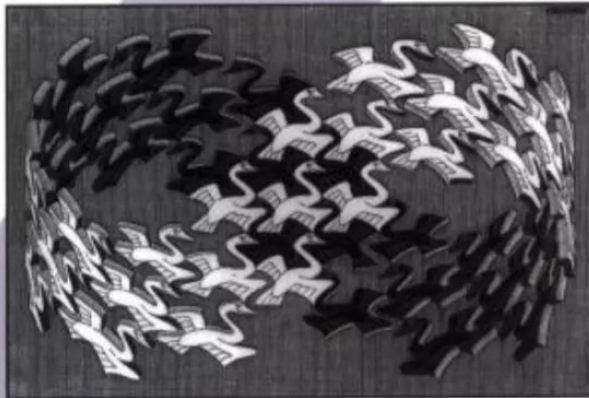
（ErichGamma）、理查德·海尔姆（Richard Helm）、拉尔夫·约翰逊（Ralph Johnson）、约翰·威利斯迪斯（John Vlissides）等 4 位作者合作出版了《设计模式：可复用面向对象软件的基础》（Design Patterns: Elements of Reusable Object-Oriented Software）一书。



Design Patterns

Elements of Reusable Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baam - Holland. All rights reserved.

Foreword by Grady Booch



在本教程中收录了23个设计模式，这是设计模式领域里程碑的事件，导致了软件设计模式的突破。



这4位作者在软件开发领域里也以他们的“四人组”（Gang of Four, GoF）匿名著称。

什么是设计模式？简单来说就是一本编程界的如来神掌。



它能提高程序员的思维能力、编程能力和设计能力。它是解决特定问题的一系列套路，有一定的普遍性。它是一套被反复使用、多数人知晓的、代码设计经验的总结。它可以提高代码的可重用性、代码的可读性和代码的可靠性以及可扩展性。它能使程序设计更加标准化，使软件开发效率提高，从而缩短软件的开发周期。是它是它就是它，编程界的武林秘籍，绝世高手都练过它！

设计模式的分类，模式分为三种不同类型的模式。

创建型模式 (Creational patterns) 5种

提供对象创建机制，增加现有代码的灵活性和重用。

类型	模式名称	学习难度	使用频率
创建型模式 Creational Pattern	单例模式 Singleton Pattern	☆☆☆☆☆	★★★★☆
	简单工厂模式 Simple Factory Pattern	☆☆☆☆☆	★★★★☆
	工厂方法模式 Factory Method Pattern	☆☆☆☆☆	★★★★★
	抽象工厂模式 Abstract Factory Pattern	★★★★☆	★★★★★
	原型模式 Prototype Pattern	★★★★☆	★★★★☆
	建造者模式 Builder Pattern	★★★★☆	★★★☆☆

结构型模式 (Structural patterns) 7种
解释如何将对象和类组装成更大的结构，同时保持结构的灵活性和高效性。

类型	模式名称	学习难度	使用频率
结构型模式 Structural Pattern	适配器模式 Adapter Pattern	☆☆☆☆☆	★★★★☆
	桥接模式 Bridge Pattern	★★★★☆	★★★★☆
	组合模式 Composite Pattern	★★★★☆	★★★★☆
	装饰模式 Decorator Pattern	★★★★☆	★★★★☆
	外观模式 Façade Pattern	☆☆☆☆☆	★★★★★
	享元模式 Flyweight Pattern	★★★★☆	★★★☆☆
	代理模式 Proxy Pattern	★★★★☆	★★★★☆

行为型模式 (Behavioral patterns) 11种
负责有效的沟通和对象之间的责任分配。

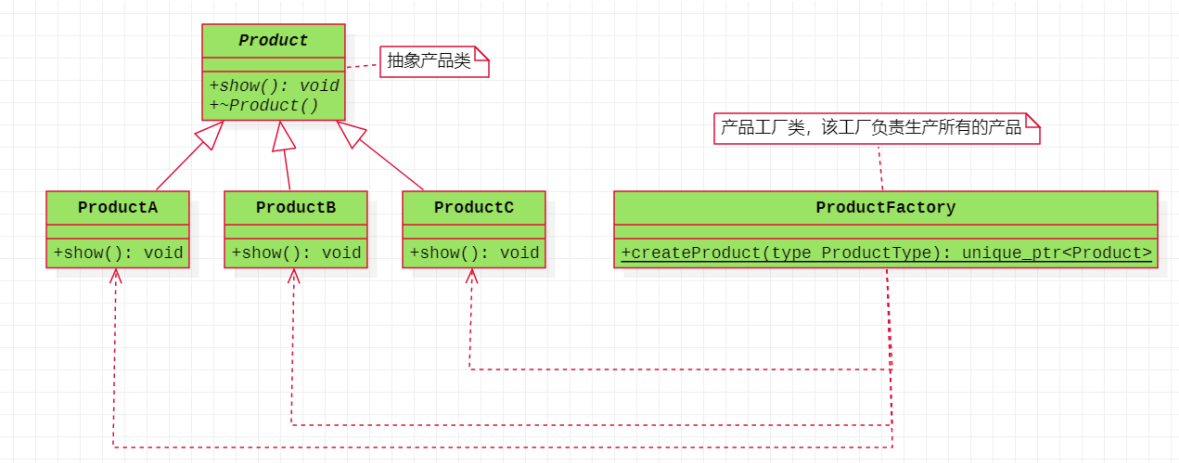
类型	模式名称	学习难度	使用频率
行为型模式 Behavioral Pattern	职责链模式 Chain of Responsibility Pattern	★★★★☆	★★★★☆
	命令模式 Command Pattern	★★★★☆	★★★★☆
	解释器模式 Interpreter Pattern	★★★★★	★★☆☆☆
	迭代器模式 Iterator Pattern	★★★★☆	★★★★★
	中介者模式 Mediator Pattern	★★★★☆	★★★★☆
	备忘录模式 Memento Pattern	★★★★☆	★★★★☆
	观察者模式 Observer Pattern	★★★★☆	★★★★★
	状态模式 State Pattern	★★★★☆	★★★★☆
	策略模式 Strategy Pattern	★★☆☆☆	★★★★☆
	模板方法模式 Template Method Pattern	★★★★☆	★★★★☆
	访问者模式 Visitor Pattern	★★★★★	★★☆☆☆

2、简单工厂

概述

简单工厂模式又叫静态工厂方法模式。提供一个工厂类，在工厂类中做判断，根据传入的类型创建相应的产品。当增加新的产品时，就需要修改工厂类。简单工厂模式提供了专门的工厂类用于创建对象，将对象的创建和对象的使用分离开，它作为一种最简单的工厂模式在软件开发中得到了较为广泛的应用。

类图



代码

```
1 enum ProductType
2 {
3     TypeA,
4     TypeB,
5     TypeC
```

```
6 };
7
8 //抽象产品类
9 class Product
10 {
11 public:
12     virtual void show() = 0;
13     //抽象类的析构函数设置为虚函数
14     virtual ~Product() {}
15 };
16
17 class ProductA
18 : public Product
19 {
20 public:
21     ProductA()
22     {
23         cout << "ProductA()" << endl;
24     }
25
26     void show() override
27     {
28         cout << "void ProductA::show()" << endl;
29     }
30
31     ~ProductA()
32     {
33         cout << "~ProductA()" << endl;
34     }
35 };
36
37 class ProductB
38 : public Product
39 {
40 public:
41     ProductB()
42     {
43         cout << "ProductB()" << endl;
44     }
45
46     void show() override
47     {
48         cout << "void ProductB::show()" << endl;
49     }
50
51     ~ProductB()
52     {
53         cout << "~ProductB()" << endl;
54     }
55 };
56
57 class ProductC
58 : public Product
59 {
60 public:
61     ProductC()
62     {
63         cout << "ProductC()" << endl;
```

```

64     }
65
66     void show() override
67     {
68         cout << "void ProductC::show()" << endl;
69     }
70
71     ~ProductC()
72     {
73         cout << "~ProductC()" << endl;
74     }
75 };
76
77 class ProductFactory
78 {
79 public:
80     static unique_ptr<Product> createProduct(ProductType type)
81     {
82         switch (type)
83         {
84             case TypeA:
85                 return unique_ptr<Product>(new ProductA());
86             case TypeB:
87                 return unique_ptr<Product>(new ProductB());
88             case TypeC:
89                 return unique_ptr<Product>(new ProductC());
90             default: return unique_ptr<Product>(nullptr);
91         }
92     }
93 };
94
95
96 void test()
97 {
98     unique_ptr<Product> pa = ProductFactory::createProduct(TypeA);
99     unique_ptr<Product> pb = ProductFactory::createProduct(TypeB);
100    unique_ptr<Product> pc = ProductFactory::createProduct(TypeC);
101    pa->show();
102    pb->show();
103    pc->show();
104 }

```

优点

- 1、工厂类包含必要的判断逻辑，可以决定在什么时候创建哪一个产品类的实例，客户端可以免除直接创建产品对象的职责，而仅仅“消费”产品，简单工厂模式实现了对象创建和使用的分离
- 2、客户端无须知道所创建的具体产品类的类名，只需要知道具体产品类所对应的参数即可，对于一些复杂的类名，通过简单工厂模式可以在一定程度减少使用者的记忆量
- 3、通过引入配置文件，可以在不修改任何客户端代码的情况下更换和增加新的具体产品类，在一定程度上提高了系统的灵活性

缺点

- 1、由于工厂类集中了所有产品的创建逻辑，职责过重，一旦不能正常工作，整个系统都要受到影响
- 2、使用简单工厂模式势必会增加系统中类的个数（引入新的工程类），增加了系统的复杂度和理解难度
- 3、系统拓展困难，一旦添加了新的产品就不得不修改工厂逻辑，在产品类型较多时，有可能造成工厂

逻辑过于复杂，不利于系统的拓展和维护

4、简单工厂模式由于使用了静态工厂方法，造成工厂角色无法形成基于继承的等级结构

使用场景

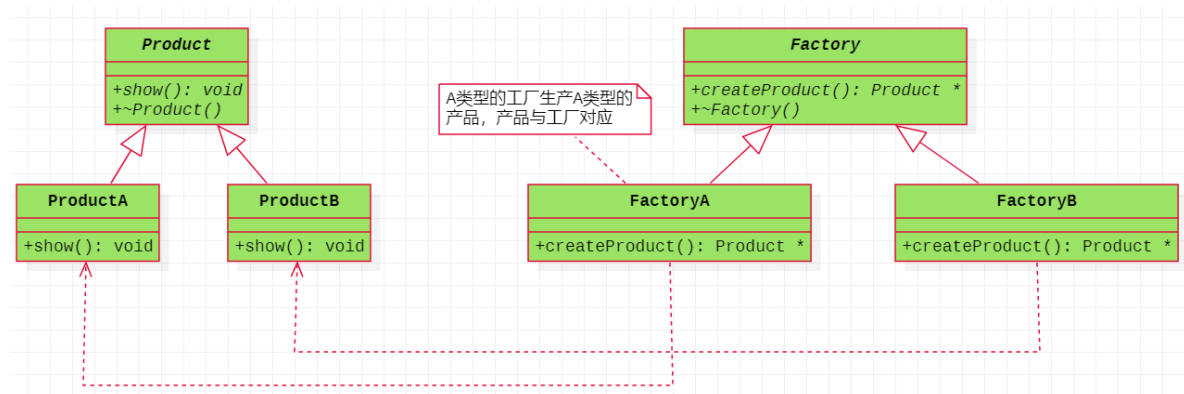
- 1、工厂类负责创建的对象比较少，由于创建的对象较少，不会造成工厂方法中的业务逻辑太过复杂。
- 2、客户端只知道传入工厂类的参数，对于如何创建对象并不关心。

2、工厂方法

概述

在软件开发及运行过程中，经常需要创建对象，但常出现由于需求的变化，需要创建的对象的具体类型也要经常变化。工厂方法通过采取虚函数的方法，实现了使用者和具体类型之间的解耦，可以用来解决这个问题。工厂方法模式对简单工厂模式中的工厂类进一步抽象。核心工厂类不再负责产品的创建，而是演变为一个抽象工厂角色，仅负责定义具体工厂子类必须实现的接口。同时，针对不同的产品提供不同的工厂。即每个产品都有一个与之对应的工厂。这样，系统在增加新产品时就不会修改工厂类逻辑而是添加新的工厂子类，从而弥补简单工厂模式对修改开放的缺陷。定义一个创建对象的接口，让子类决定实例化哪个类。该模式使类对象的创建延迟到子类。

类图



代码

```
1 //产品抽象类
2 class Product
3 {
4     public:
5         virtual void show() = 0;
6         virtual ~Product() {}
7 };
8
9 class ProductA
10 : public Product
11 {
12     public:
13         ProductA()
14         {
15             cout << "ProductA()" << endl;
16         }
17
18         void show() override
19         {
20             cout << "void ProductA::show()" << endl;
```

```

21     }
22
23     ~ProductA()
24     {
25         cout << "~ProductA()" << endl;
26     }
27
28 };
29
30 class ProductB
31 : public Product
32 {
33 public:
34     ProductB()
35     {
36         cout << "ProductB()" << endl;
37     }
38
39     void show() override
40     {
41         cout << "void ProductB::show()" << endl;
42     }
43
44     ~ProductB()
45     {
46         cout << "~ProductB()" << endl;
47     }
48
49 };
50
51 class Factory
52 {
53 public:
54     virtual Product* createProduct() = 0;
55     virtual ~Factory() {}
56 };
57
58 class FactoryA
59 : public Factory
60 {
61 public:
62     FactoryA()
63     {
64         cout << "FactoryA()" << endl;
65     }
66
67     Product* createProduct() override
68     {
69         return new ProductA();
70     }
71
72     ~FactoryA()
73     {
74         cout << "~FactoryA()" << endl;
75     }
76 };
77
78 class FactoryB

```

```

79 : public Factory
80 {
81 public:
82     FactoryB()
83     {
84         cout << "FactoryB()" << endl;
85     }
86
87     Product* createProduct() override
88     {
89         return new ProductB();
90     }
91
92     ~FactoryB()
93     {
94         cout << "~FactoryB()" << endl;
95     }
96 };
97
98
99 void test()
100 {
101     //生产产品A
102     unique_ptr<Factory> factoryA(new FactoryA());
103     unique_ptr<Product> productA(factoryA->createProduct());
104     productA->show();
105
106     //生产产品B
107     cout << endl << endl;
108     unique_ptr<Factory> factoryB(new FactoryB());
109     unique_ptr<Product> productB(factoryB->createProduct());
110     productB->show();
111 }

```

优点

- 1、用户只需要知道具体工厂的名称就可得到所要的产品，无须知道产品的具体创建过程。
- 2、灵活性增强，对于新产品的创建，只需多写一个相应的工厂类。
- 3、典型的解耦框架。高层模块只需要知道产品的抽象类，无须关心其他实现类，满足迪米特法则、依赖倒置原则和里氏替换原则。
- 4、对扩展开放对修改关闭；解决了简单工厂的缺点问题。

缺点

- 1、类的个数容易过多，增加复杂度
- 2、增加了系统的抽象性和理解难度
- 3、接口的传入参数类型需要一致，且只能对单一变化量接口使用
- 4、抽象产品只能生产一种产品，此弊端可使用抽象工厂模式解决。

应用场景

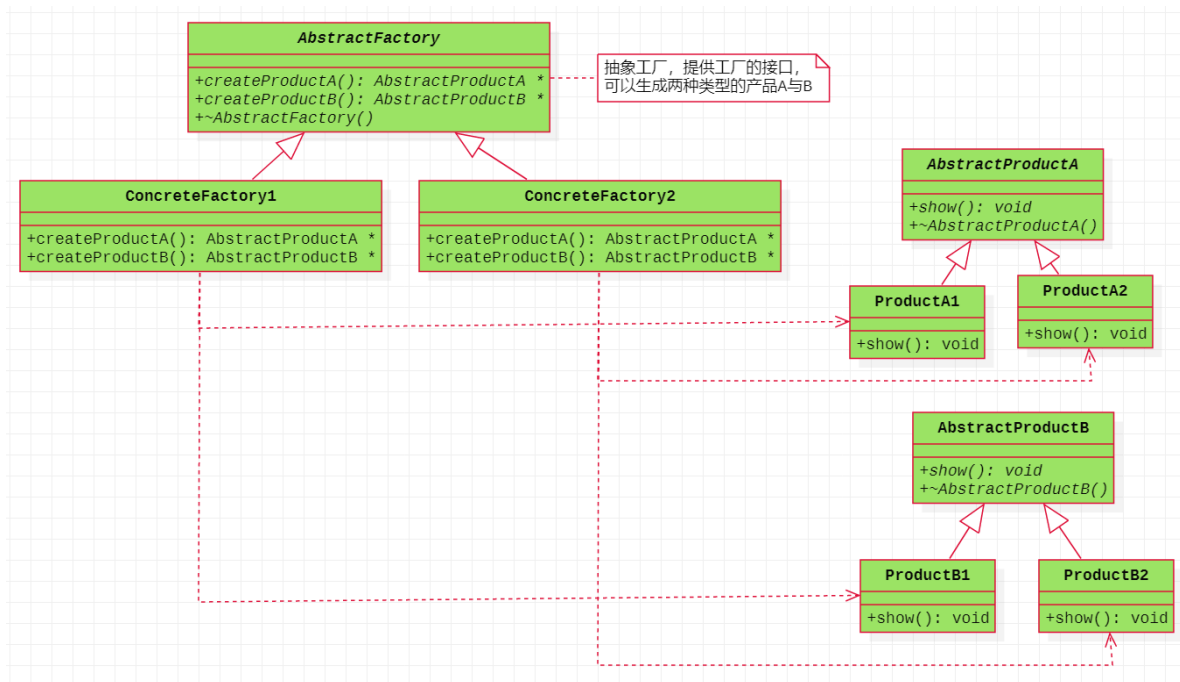
- 1、客户只知道创建产品的工厂名，而不知道具体的产品名。
- 2、创建对象的任务由多个具体子工厂中的某一个完成，而抽象工厂只提供创建产品的接口。
- 3、客户不关心创建产品的细节，只关心产品的品牌

3、抽象工厂

在软件开发及运行过程中，经常面临着“一系列相互依赖的对象”的创建工作；而由于需求的变化，常常存在更多系列对象的创建问题。

定义：提供一个接口，该接口负责创建一系列“相关或者相互依赖的对象”，无需指定它们具体的类。

类图



代码

```
1 //A类型的抽象产品
2 class AbstractProductA
3 {
4 public:
5     virtual void show() = 0;
6     ~AbstractProductA(){}
7 };
8
9 //B类型的抽象产品
10 class AbstractProductB
11 {
12 public:
13     virtual void show() = 0;
14     ~AbstractProductB(){}
15 };
16
17 class ProductA1
18 : public AbstractProductA
19 {
20 public:
21     virtual void show() override
22     {
23         cout << "void ProductA1::show()" << endl;
24     }
25 };
26
27 class ProductA2
28 : public AbstractProductA
```

```

29 {
30     public:
31         virtual void show() override
32         {
33             cout << "void ProductA2::show()" << endl;
34         }
35 };
36
37 class ProductB1
38 : public AbstractProductB
39 {
40     public:
41         virtual void show() override
42         {
43             cout << "void ProductB1::show()" << endl;
44         }
45 };
46
47 class ProductB2
48 : public AbstractProductB
49 {
50     public:
51         virtual void show() override
52         {
53             cout << "void ProductB2::show()" << endl;
54         }
55 };
56
57 class AbstractFactory
58 {
59     public:
60         virtual AbstractProductA *createProductA() = 0;
61         virtual AbstractProductB *createProductB() = 0;
62         ~AbstractFactory() {}
63 };
64
65 class ConcreteFactory1
66 : public AbstractFactory
67 {
68     public:
69         AbstractProductA *createProductA() override
70         {
71             return new ProductA1();
72         }
73
74         AbstractProductB *createProductB() override
75         {
76             return new ProductB1();
77         }
78
79 };
80
81 class ConcreteFactory2
82 : public AbstractFactory
83 {
84     public:
85         AbstractProductA *createProductA() override
86         {

```

```

87         return new ProductA2();
88     }
89
90     AbstractProductB *createProductB() override
91     {
92         return new ProductB2();
93     }
94
95 };
96
97 void test()
98 {
99     AbstractFactory *factory1 = new ConcreteFactory1();
100     AbstractProductA *productA = factory1->createProductA();
101     AbstractProductB *productB = factory1->createProductB();
102     productA->show();
103     productB->show();
104
105     cout << endl;
106     AbstractFactory *factory2 = new ConcreteFactory2();
107     productA = factory2->createProductA();
108     productB = factory2->createProductB();
109     productA->show();
110     productB->show();
111 }
112

```

优点

- 1、抽象工厂模式隔离了具体类的生成，使得客户端并不需要知道什么被创建。
- 2、当一个产品族中的多个对象被设计成一起工作时，它能够保证客户端始终只使用同一产品族中的对象；
- 3、增加新的产品族很方便（生成新的具体工厂），无需修改已有系统代码，符合开闭原则；

缺点

增加新的产品等级结构很复杂，需要修改抽象工厂和所有的具体工厂类，对“开闭原则”的支持呈现倾斜性。

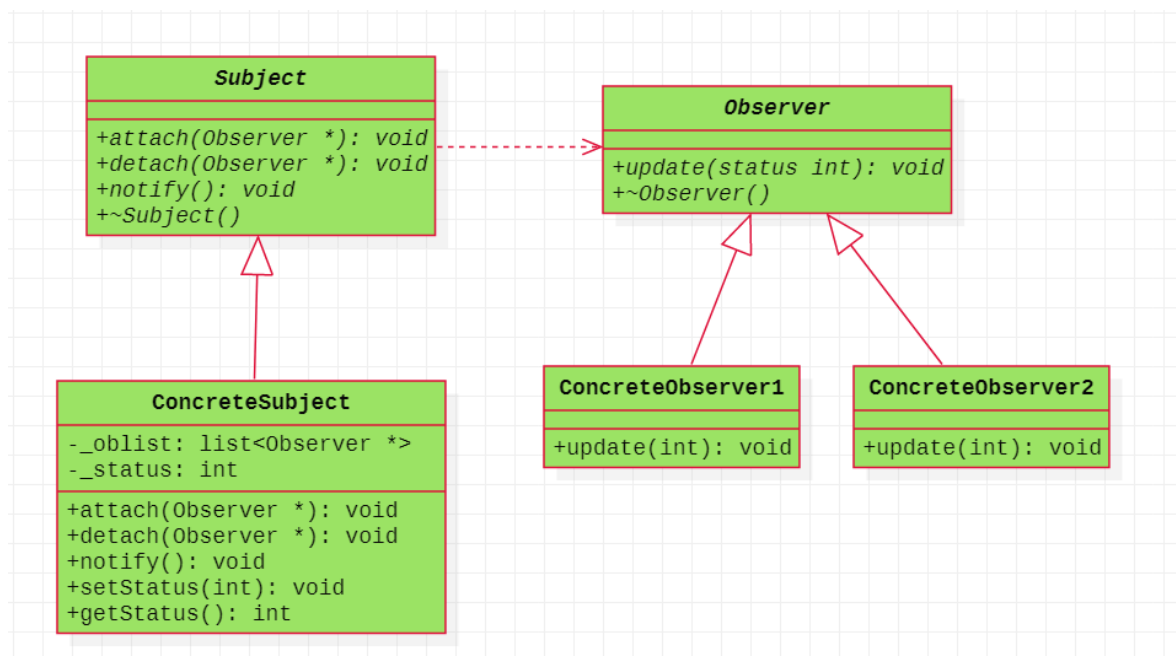
应用场景

- 1、用户无需关心对象的创建过程，将对象的创建和使用解耦；
- 2、产品等级结构稳定，在设计完成之后不会向系统中增加新的产品等级结构或者删除已有的产品等级结构；
- 3、系统中有多于一个的产品族，而每次只使用其中某一产品族。可以通过配置文件等方式来使用户能够动态改变产品族，也可以很方便的增加新的产品族

4、观察者模式

在GOF的《设计模式：可复用面向对象软件的基础》一书中对观察者模式是这样定义的：定义对象的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。当一个对象发生了变化，关注它的对象就会得到通知；这种交互也成为发布-订阅（publish-subscribe）。

类图



代码

```

1  #include <iostream>
2  #include <list>
3  #include <algorithm>
4  #include <string>
5
6  using std::cout;
7  using std::endl;
8  using std::list;
9  using std::find;
10 using std::string;
11
12 class Observer;
13
14 class Subject
15 {
16 public:
17     virtual void attach(Observer *pobserver) = 0;
18     virtual void detach(Observer *pobserver) = 0;
19     virtual void notify() = 0;
20
21     virtual ~Subject() {}
22 };
23
24 class ConcreteSubject
25 : public Subject
26 {
27 public:
28     void attach(Observer *pobserver) override;
29     void detach(Observer *pobserver) override;
30     void notify() override;
31
32     void setStatus(int status)
33     {
34         _status = status;
35     }
36

```

```

37     int getStatus() const
38     {
39         return _status;
40     }
41
42 private:
43     list<Observer *> _obList;
44     int _status;
45
46 };
47
48 class Observer
49 {
50 public:
51     virtual void update(int) = 0;
52     virtual ~Observer() { }
53 };
54
55 class ConcreteObserver
56 : public Observer
57 {
58 public:
59     ConcreteObserver(const string &name)
60         : _name(name)
61     {
62
63     }
64     void update(int value)
65     {
66         cout << "ConcreteObserver " << _name << ", value = " << value <<
endl;
67     }
68
69 private:
70     string _name;
71 };
72
73 class ConcreteObserver2
74 : public Observer
75 {
76 public:
77     ConcreteObserver2(const string &name)
78         : _name(name)
79     {
80
81     }
82     void update(int value)
83     {
84         cout << "ConcreteObserver2 " << _name << ", value = " << value <<
endl;
85     }
86
87 private:
88     string _name;
89
90 };
91
92 void ConcreteSubject::attach(Observer *pobserver)

```

```

93 {
94     _obList.push_back(pobserver);
95 }
96
97 void ConcreteSubject::detach(Observer *pobserver)
98 {
99     for(auto it = _obList.begin(); it != _obList.end(); ++it)
100     {
101         if(*it == pobserver)
102         {
103             _obList.remove(pobserver);
104             break;
105         }
106     }
107 }
108
109 void ConcreteSubject::notify()
110 {
111     for(auto &ob : _obList)
112     {
113         ob->update(_status);
114     }
115 }
116
117 void test()
118 {
119     unique_ptr<ConcreteSubject> pSubject(new ConcreteSubject());
120     unique_ptr<Observer> pobserver(new ConcreteObserver("lili"));
121     unique_ptr<Observer> pobserver2(new ConcreteObserver2("lucy"));
122
123     pSubject->setStatus(2);
124
125     pSubject->attach(pobserver.get());
126     pSubject->attach(pobserver2.get());
127
128     pSubject->notify();
129
130     pSubject->detach(pobserver2.get());
131     pSubject->setStatus(3);
132     pSubject->notify();
133 }

```

优点

- 1、观察者和被观察者是抽象耦合的
- 2、建立一套触发机制。

缺点

- 1、如果一个被观察者对象有很多的直接和间接的观察者的话，将所有的观察者都通知到会花费很多时间。
- 2、如果在观察者和观察目标之间有循环依赖的话，观察目标会触发它们之间进行循环调用，可能导致系统崩溃。
- 3、观察者模式没有相应的机制让观察者知道所观察的目标对象是怎么发生变化的，而仅仅只是知道观察目标发生了变化。

应用场景

- 1、一个抽象模型有两个方面，其中一个方面依赖于另外一个方面。将这些方面封装在独立的对象中使它们可以各自独立地改变和复用。
- 2、一个对象的改变将导致其它一个或多个对象发生改变，而不知道具体有多少对象将发生改变，可以降低对象之间的耦合度。
- 3、一个对象必须通知其他对象，而并不知道这些对象是谁。
- 4、需要在系统中创建一个触发链，A对象的行为将影响B对象，B对象的行为将影响C对象.....，可以使用观察者模式创建一种链式触发机制。